CHAPTER 5: Operating Systems

Question 1: How a single CPU appears to run many processes simultaneously

A single CPU core achieves the illusion of multitasking through time-division multiplexing (Preemptive Multitasking). The OS kernel's scheduler divides CPU time ~~into~~ into microscopic time slices (quanta, typically 1 to 20 milliseconds). When a process's quantum expires, a hardware timer triggers an interrupt. The OS halts the current process, performs a context switch (saving CPU registers, program counter, and stack pointer to memory), and loads the state of the next ready process. Because these switches occur hundreds of times per second, human perception experiences continuous, simultaneous execution.

Question 2: Why Chrome cannot directly access VS Code's memory

The OS enforces memory protection and isolation through hardware-backed Virtual Memory Management (MMU). Each running process is given its own private, isolated virtual address space.

Security & Stability: If Chrome could read VS Code's memory, malicious web scripts or compromised browser tabs could inspect your private code, environment secrets, or session tokens.

Fault Isolation: If Chrome crashes, dereferences a null pointer, or suffers a buffer overflow, it only destroys its own address space without corrupting VS Code or bringing down the entire operating system.

Question 3: Memory Allocation and Process Memory Layout

When an OS loads an executable into RAM, it assigns it a contiguous virtual address space divided into distinct segments:

Text Segment (Code): Read-only machine instructions compiled from the source code.

Data Segment (Initialized Data): Global and static variables with initial values.

BSS Segment (Uninitialized Data): Global and static variables initialized to zero by default.

Heap: Dynamically allocated memory requested at runtime (e.g., malloc(), new), which grows upward toward higher addresses.

Stack: Automatically managed memory for local function variables, call frames, return addresses, and parameter passing. It grows downward toward the heap.

Question 4: The OS and Your College Operations Team

Responsibility 1: Memory Allocation & Separation vs. Room Assignment

OS Function: Allocates isolated memory regions to processes to prevent mutual overwrites.
College Analogy: The operations team assigns specific classrooms and lecture halls to scheduled sections so two different batches do not collide in the same space.
If Removed: In an OS, processes would overwrite each other's data causing immediate system crashes. In college, multiple professors and cohorts would show up to room 101 simultaneously, leading to total scheduling paralysis.
Responsibility 2: Device Management & I/O Abstraction vs. Classroom AV/AC Infrastructure
OS Function: Device drivers handle low-level communication with monitors, printers, and disks so code can just call print().
College Analogy: Operations manages projectors, power sockets, AC units, and lighting so teachers can walk in, plug in an HDMI cable, and begin lecturing immediately.
If Removed: In an OS, every software developer would have to ~~write~~ write custom hardware drivers for every screen model. In college, professors would have to climb ladders, repair projector fuses, and balance room voltage before teaching.
Responsibility 3: CPU Scheduling vs. Shared Campus Facilities Scheduling
OS Function: The kernel scheduler allocates time slices for shared CPU cores based on priority and readiness.
College Analogy: Operations schedules access to the sports arena, auditorium, conference ~~hotel~~ halls, and labs among different student clubs and academic departments.
If Removed: In an ~~SO~~ OS, one greedy process would monopolize the processor, freezing the entire machine. In college, one aggressive club would permanently take over the auditorium, preventing all other campus events.